Software Reviews, Reliability & Quality Standards

Software Reviews – Introduction

- Systematic examination of software artifacts
- Detect defects early
- Improve quality
- Reduce cost of rework

Objectives of Software Reviews

- Find defects before testing
- Verify compliance with requirements
- Improve maintainability
- Ensure coding & design standards followed

Types of Software Reviews

- Informal Review
- Walkthrough
- Technical Review
- Formal Technical Review (FTR)
- Inspection

Definition

Informal Review is a simple review process where software documents or code are examined without a structured procedure or formal documentation.

It is the **most basic type of software review** and does not follow strict rules.

Key Characteristics

- No formal meeting required
- No strict documentation
- No defined roles (optional)
- Flexible and quick process
- Used for early defect detection

Purpose

- Find errors quickly
- Improve quality
- Share knowledge among team members
- Clarify requirements or logic

Example of Walkthrough

Example Scenario: Login System

A developer writes the following requirement:
“System should validate user quickly.”

During walkthrough:
Team members say:
•What does "quickly" mean?
•Is there any time limit?
•What happens if password is wrong?

After discussion, requirement is updated to:

“System should validate user credentials within 2 seconds and display an error message for invalid password.”

Ambiguity removed.
Requirement improved.
Future coding errors prevented.

Technical Review (Software Review)

Definition

A **Technical Review** is a structured peer review process in which software documents, design, or code are examined by technical experts to detect defects and ensure correctness.

It is more formal than a walkthrough but less rigid than an inspection.

Objectives of Technical Review

- Detect errors in design, requirements, or code
- Verify compliance with standards
- Ensure technical correctness
- Improve software quality
- Reduce future maintenance cost

Characteristics

- Conducted by technical experts
- Focus on technical aspects (logic, algorithms, design)
- May include documentation
- Defects are recorded
- Moderated meeting

Participants

1. **Moderator** – Conducts the review
2. **Author** – Creator of the document/code
3. **Reviewers** – Technical team members
4. **Recorder** – Notes defects and suggestions

Technical Review Process

1. Planning
2. Preparation
3. Review Meeting
4. Rework
5. Follow-up

Example: ATM Withdrawal Module

A developer designs an ATM withdrawal function.

During technical review, reviewers find:

- No validation for insufficient balance
- No error handling for network failure
- Missing daily withdrawal limit check

After review, developer updates the design to include:

- Balance verification
- Error handling
- Limit validation

👉 Result: Improved reliability and reduced future failures.

Advantages

✓ Early defect detection
✓ Improved design quality
✓ Knowledge sharing
✓ Reduced development cost

Formal Technical Review (FTR)

- Structured and planned review meeting
- Focus on defect detection
- Checklist-based evaluation
- Produces documented review report

Participants in FTR

- Moderator – Leads review
- Author – Developer of document/code
- Reviewer – Finds defects
- Recorder – Documents issues

FTR Process Steps

- Planning
- Preparation
- Review Meeting
- Rework
- Follow-up

Benefits of Formal Reviews

- Early defect detection
- Improved software quality
- Knowledge sharing
- Reduced development cost

Software Reliability – Definition

- Software reliability is the ability of software to perform its required functions without failure under given conditions for a given time period.

Reliability vs Availability

- Reliability – No failure during operation
- Availability – System ready when needed

Reliability Metrics

- **Reliability Metrics with Examples**
- Reliability metrics are used to measure how reliable a software system is. They help us understand how often the software fails, how long it works without failure, and how quickly it can be repaired.
- **1. POFOD (Probability of Failure on Demand)**
- POFOD means the probability that the system will fail when a service is requested.
- **Example:**
  Suppose an emergency shutdown system is used 200 times, and it fails 4 times.
- **POFOD = 4 / 200 = 0.02**
- This means there is a 2% chance of failure whenever the system is demanded.

2. ROCOF (Rate of Occurrence of Failure)

ROCOF shows how often failures occur in the system over a period of time.

Example:

If a software system fails 6 times in 12 hours, then

ROCOF = 6 / 12 = 0.5 failures per hour

This means the software fails at a rate of 0.5 times per hour.

3. MTTF (Mean Time To Failure)

MTTF is the average time the software works correctly before a failure happens.

Formula:

MTTF = Total operating time / Number of failures

Example:

If software runs for 300 hours and fails 5 times, then

MTTF = 300 / 5 = 60 hours

This means the software works for 60 hours on average before failure.

4. MTTR (Mean Time To Repair)

MTTR is the average time needed to fix the software after a failure.

Formula:

MTTR = Total repair time / Number of repairs

Example:

If total repair time for 5 failures is 10 hours, then

MTTR = 10 / 5 = 2 hours

This means each failure takes 2 hours on average to repair.

5. MTBF (Mean Time Between Failures)

MTBF is the average time between two consecutive failures. It includes both working time and repair time.

Formula:

MTBF = MTTF + MTTR

Example:

If

MTTF = 60 hours

and

MTTR = 2 hours

Then

MTBF = 60 + 2 = 62 hours

This means the average time between two failures is 62 hours.

Improving Software Reliability

- Software reliability can be improved by using better development and testing practices. The main ways to improve software reliability are:Software reliability can be improved by avoiding faults, detecting and removing errors, using fault tolerance, handling input properly, and doing regular maintenance.
- **1. Fault Avoidance**
- Fault avoidance means preventing faults from being introduced into the software during development.
- **Example:**
  Using proper coding standards, design methods, and developer training helps reduce mistakes in the software.

2. Fault Detection and Removal

This method focuses on finding and fixing faults before the software is released.

Example:

Code reviews, testing, debugging, and inspections help detect errors and remove them early.

3. Fault Tolerance

Fault tolerance means the software continues to work even if some faults are present.

Example:

A banking system may save backup data and continue processing even if one server fails.

4. Robust Input Handling

Software should handle invalid or unexpected input properly.

Example:

If a user enters letters instead of numbers, the system should show an error message instead of crashing.

5. Reliable Design and Architecture

A good software design reduces complexity and makes the system more dependable.

Example:

Dividing a large system into smaller modules makes it easier to test and maintain.

6. Regular Maintenance and Updates

Software reliability improves when bugs are fixed and updates are applied regularly.

Example:

Developers release patches to fix security issues and performance problems.

Quality Standards – Overview

- **Quality Standards – Overview**
- Quality standards are a set of rules, guidelines, and best practices used to ensure that software products and software processes maintain a required level of quality. These standards help organizations develop reliable, efficient, maintainable, and user-friendly software.
- In software engineering, quality standards are important because they provide a common framework for development, testing, maintenance, and management. They improve consistency, reduce errors, increase customer satisfaction, and help organizations produce better software.

Why Quality Standards are Important

- Quality standards are important for many reasons:
- They improve the quality of software products.
- They help in reducing defects and failures.
- They ensure that software development follows a systematic process.
- They increase customer confidence and satisfaction.
- They support continuous improvement in software processes.
- They make software easier to maintain and upgrade.

Main Quality Standards in Software Engineering

- Some important quality standards used in software engineering are:
- **1. ISO 9000**
- ISO 9000 is a family of international standards related to quality management systems. It focuses on improving organizational processes and ensuring that products and services consistently meet customer requirements.
- In software engineering, ISO 9000 helps organizations establish a proper quality management system. It ensures that every stage of development is controlled and documented.

Main points of ISO 9000:

- Focus on quality management
- Customer satisfaction
- Proper documentation
- Process control
- Continuous improvement

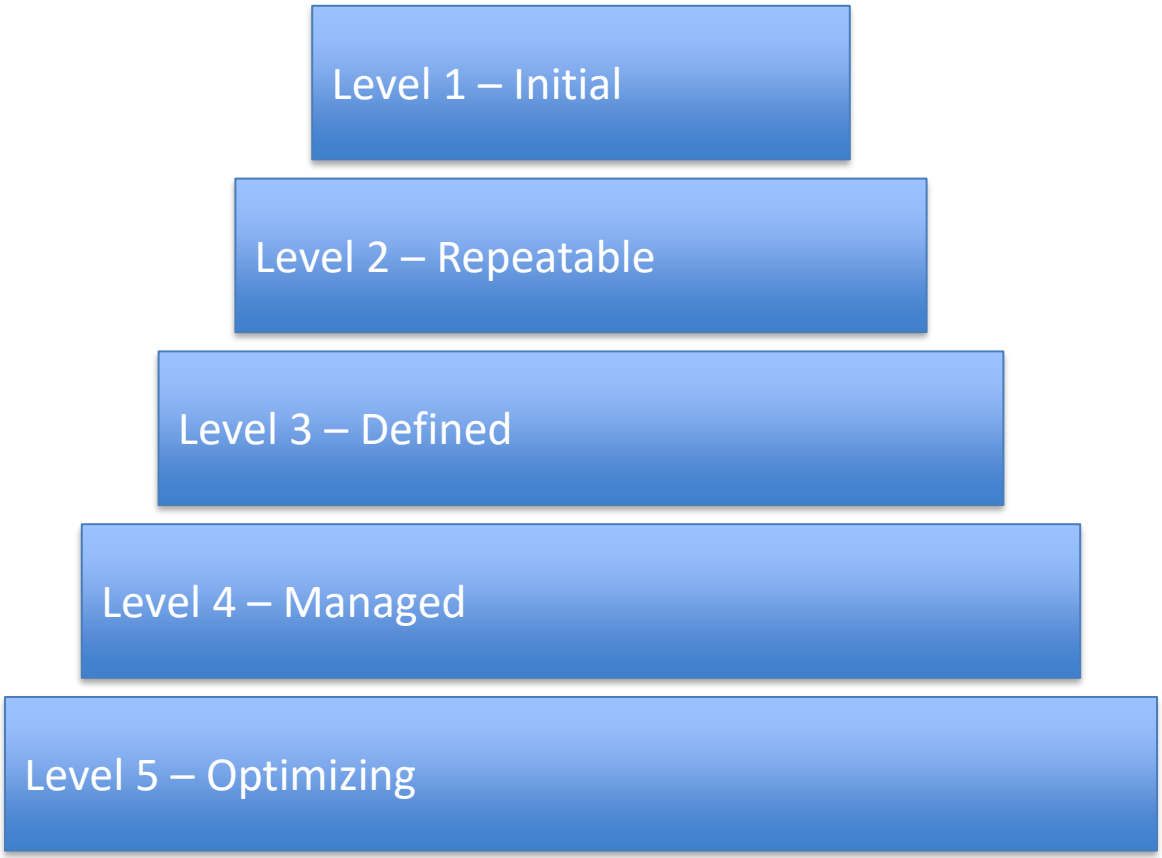
Example:

A software company following ISO 9000 maintains clear documentation, testing procedures, and customer feedback systems to improve product quality.

2. CMM (Capability Maturity Model)

- Developed by SEI
- Improves software process capability
- Focus on process maturity

CMM Pyramid


Level 1 – Initial
Level 2 – Repeatable
Level 3 – Defined
Level 4 – Managed
Level 5 – Optimizing

CMM Maturity Levels

- CMM is a framework used to measure the maturity of an organization's software development process. It tells how well the software process is defined, managed, measured, and improved.
- CMM has five maturity levels:
- **Level 1 – Initial:**
  Process is unorganized and unpredictable.
- **Level 2 – Repeatable:**
  Basic project management practices are followed.
- **Level 3 – Defined:**
  Processes are documented and standardized.
- **Level 4 – Managed:**
  Processes are measured and controlled.
- **Level 5 – Optimizing:**
  Continuous process improvement is done.
- **Example:**
  A company at CMM Level 1 may develop software without proper planning, while a company at Level 5 continuously improves its development process using data and feedback.

Six Sigma for Software Engineering

- Six Sigma is a quality improvement methodology used to reduce defects and improve process performance. It uses statistical methods to identify and remove causes of errors.
- The main goal of Six Sigma is to make processes nearly defect-free.
- In software engineering, Six Sigma is used to improve development, testing, and maintenance processes.

Example:

If a softwa**DMAIC steps in Six Sigma:**

- **D – Define**
- **M – Measure**
- **A – Analyze**
- **I – Improve**
- **C – Control**

re company finds many defects in the testing phase, Six Sigma helps analyze the root causes and improve the process to reduce future defects.

1. Define

In this step, the problem is clearly identified.

The goal of the project, customer requirements, and process boundaries are defined.

What is done in Define step?

- Identify the problem
- Define the project goal
- Understand customer needs
- Identify the process to be improved
- Define team roles and scope

Example

Suppose a software company finds that many customers are complaining about **late bug fixes** in a mobile application.

In the **Define** phase, the company writes the problem as:

“Bug fixing in the mobile app is taking too much time, causing customer dissatisfaction.”

The goal may be:

“Reduce average bug fixing time from 10 days to 4 days.”

2. Measure

In this step, data is collected to understand the current performance of the process.

This helps in knowing how serious the problem is.

What is done in Measure step?

•Collect relevant data

•Measure current process performance

•Identify defects, delays, or errors

•Establish baseline values

Example

The software company collects data for the last 3 months and finds:

•Average bug fixing time = 10 days

•High-priority bugs = 25 per month

•40% of bugs are delayed

•Testing rework is very high

This data shows the current condition of the process.

3. Analyze

In this step, the root causes of the problem are identified.

The team studies the collected data to find why the problem is happening.

What is done in Analyze step?

•Identify root causes

•Study process weaknesses

•Find where delays or defects occur

•Use tools like cause-and-effect analysis

Example

After analysis, the company finds these root causes:

•Requirements are not clearly written

•Developers are not getting bug details properly

•Communication gap between testing and development teams

•No proper priority system for fixing bugs

So, the real reason is not only delay, but poor process management.

4. Improve

In this step, solutions are developed and implemented to remove the root causes.
The process is modified to achieve better results.

What is done in Improve step?

- Suggest solutions
- Remove root causes
- Improve workflow
- Test and implement changes

Example

The company takes these actions:

- Introduces a standard bug report format
- Starts daily coordination meetings between testers and developers
- Sets bug priority levels: high, medium, low
- Gives training on requirement documentation

After applying these improvements, average bug fixing time reduces from **10 days to 5 days**.

5. Control

In this step, the improved process is monitored and controlled so that the problem does not return in future.

What is done in Control step?

•Monitor the improved process
•Create control plans
•Set performance standards
•Ensure improvements continue

Example

The company now:

•Reviews bug fixing reports every week
•Uses dashboards to track bug resolution time
•Conducts monthly quality meetings
•Ensures the new bug reporting format is always followed

This helps maintain the improved process and prevents old problems from coming back.

4. IEEE Standards

- IEEE provides standards for software requirements, design, testing, maintenance, and documentation. These standards help software engineers follow consistent methods while developing software.
- **Example:**
  IEEE standards can be used for writing Software Requirement Specifications (SRS) in a proper format.

5. Software Reliability Standards

- These standards focus on making software dependable and reducing system failure. They ensure that the software performs correctly under specified conditions for a certain period.
- **Example:**
  Testing a medical software system thoroughly to ensure it works correctly without failure is part of reliability-focused standards.

Benefits of Quality Standards

- Quality standards provide many benefits:
- Better software quality
- Fewer defects
- Improved reliability
- Better documentation
- Easier maintenance
- Higher customer trust
- Better team coordination
- Continuous process improvement